

# Лабораторная работа 10

## ДАМПирование Памяти

### Задача лабораторной работы:

Требуется скорректировать программу (и соответствующие файлы) так, чтобы экранная строка дампа содержала бы сначала 4-х позиционный шестнадцатеричный адрес первого байта строки, далее один пробел, после которого шестнадцатеричное представление 16-и байтов памяти. Далее следует еще один пробел, после которого следует символьное представление этих же 16-и байтов (между символами пробелов нет). При этом требуется скорректировать процедуру `Write_char` так, чтобы вместо любого управляющего символа (код ASCII от 00h до 1Fh включительно) должен выводиться символ ".". Пример экранной строки: A17F 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFG. .АБВГДЕ 146

Примечание 1. Переменная-слово `Address` содержит начальный адрес смещения 256-и байтовой области (см. рис.42). Поэтому начальный адрес строки рассчитывается в процедуре `Disp_line` путем суммирования содержимого `Address` с содержимым регистра `DX`. Это суммирование выполняет оператор: `add dx, [Address]` Обратите внимание, что имя переменной `Address` заключено в квадратные скобки. При отсутствии этих скобок к содержимому регистра `dx` было бы прибавлено не содержимое переменной, а ее адрес-смещение. Не забудьте определить переменную-слово `Address` в начале файла `Disp_sec.asm`, задав ей первоначальное значение, например 100h. Для этого используется псевдооператор резервирования слова памяти `dw`: `Address dw 100h` В последующих работах будут созданы процедуры, которые загружают в переменную `Sector` редактируемый фрагмент ОП, а в переменную `Address` – начальный адрес (внутрисегментное смещение) этого фрагмента.

Примечание 2. Перед оператором с меткой ".М" в `Disp_line` запишите оператор `"push bx"`, который сохранит в стеке номер первого байта выводимой строки. Этот номер пригодится при выводе на экран символов (для его получения не забудьте записать оператор `"pop bx"`).

### Ход работы:

## Файл source.asm:

```
%include "write_word.asm"
%include "write_byte.asm"
%include "write_char.asm"
%include "write_address.asm"
%include "write_digit.asm"

section .data
    word_data dw 3039h ; число 12345 в двоичном представлении

section .bss

section .text
    global _start

_start:
    movzx rdx, word_data
    call Test_write_word_dec
    movzx rdx, word_data
    call Test_write_word_hex

; Завершение программы
mov eax, 60 ; Системный вызов для выхода из программы
xor edi, edi ; Код возврата 0
syscall

Test_write_word_dec:
    movzx rdx, word_data
    call Write_word_dec
    ret

Test_write_word_hex:
    movzx rdx, word_data
    call Write_word_hex
    ret
```

## Файл write\_word.asm

```
%include "write_byte.asm"
%include "write_char.asm"
%include "write_address.asm"

Write_word_dec:
; Сохранение регистров, используемых в процедуре
push rax
push rax

; Инициализация
mov rcx, 0 ; Счетчик цифр
mov rax, rdx ; Перенос числа в регистр RAX
mov rbx, 10 ; Основание системы счисления

.divide_loop:
xor rdx, rdx ; Очистка регистра RDX перед делением
div rbx ; RAX / RBX, результат в RAX, остаток в RDX
push rdx ; Сохранение остатка на стеке
inc rcx ; Увеличение счетчика цифр
test rax, rax ; Проверка, если RAX равно 0
jne .divide_loop ; Продолжение цикла, если RAX не равно 0

.print_loop:
pop rdx ; Извлечение остатка из стека
add di, '0' ; Преобразование в ASCII
call Write_char ; Вывод символа
loop .print_loop ; Повторение для всех цифр

; Восстановление регистров
pop rax
pop rax
ret

Write_word_hex:
; Сохранение регистров, используемых в процедуре
push rax
push rdx

; Вывод адреса
mov rax, rdx ; Загрузка адреса в RAX
call Write_address_hex

; Вывод шестнадцатеричного представления
movzx rax, dx
call Write_byte_hex
movzx rax, di
call Write_byte_hex

; Вывод символического представления
movzx rax, dx
call Write_char_representation
movzx rax, di
call Write_char_representation

; Восстановление регистров
pop rdx
pop rax
ret
```

Файл write\_byte.asm:

```
%include "write_digit.asm"
%include "write_char.asm"

Write_byte_hex:
    push rbp
    mov rbp, 16      ; Основание системы счисления

    ; Вывод старшего полубайта
    mov al, di
    shr al, 4        ; Сдвиг на 4 бита вправо
    call Write_digit_hex

    ; Вывод младшего полубайта
    mov al, di
    and al, 0Fh      ; Маскирование старших 4 бит
    call Write_digit_hex

    pop rbp
    ret
```

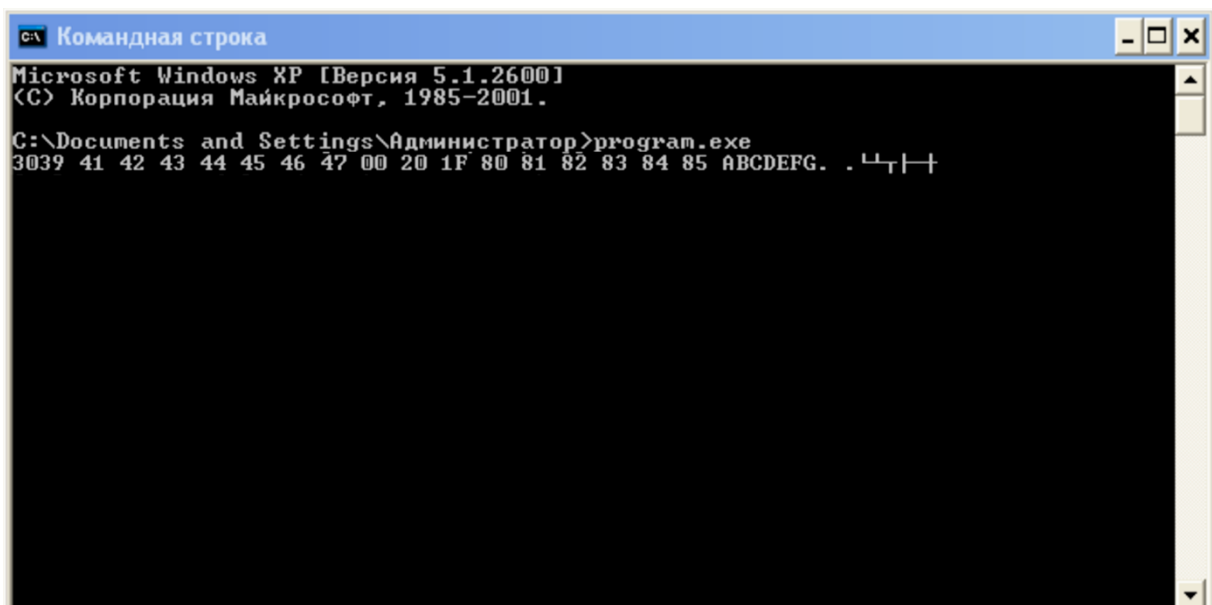
Файл write\_char.asm:

```
%include "write_byte.asm"

Write_char:
    cmp al, 20h      ; Проверка, является ли символ пробелом или знаком перевода строки
    je .check_special_characters

    mov eax, 1       ; Системный вызов для вывода
    mov edi, 1       ; дескриптор stdout
    mov rsi, rsp      ; Адрес символа
    mov edx, 1       ; Длинна
```

Результат работы программы:



```
Microsoft Windows XP [Версия 5.1.2600]
(C) Корпорация Майкрософт, 1985-2001.

C:\Documents and Settings\Администратор>program.exe
3039 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFGH. .┌─┐└─┘
```